

Data Structures in R & Python

Lecture 06

Dr. Colin Rundel

Tuples

Tuples

Python tuples are *heterogeneous, ordered, immutable* containers - lists whose contents cannot be changed. R has no tuple type, a list (or an atomic vector) is used instead.

```
1 (1, 2, 3)
```

py

```
(1, 2, 3)
```

```
1 (1, True, "abc")
```

py

```
(1, True, 'abc')
```

```
1 type( (1) )
```

py

```
<class 'int'>
```

```
1 type( (1,) )
```

py

```
<class 'tuple'>
```

```
1 x = (1, 2, 3)
```

py

```
2 x[2] = 5
```

TypeError: 'tuple' object does not support

```
1 x = (1, [2, 3])
```

py

```
2 x[1].append(4)
```

```
3 x
```

```
(1, [2, 3, 4])
```

It is the *comma* that creates a tuple, not the parentheses - (1) is just the number 1 and a one element tuple must be

Unpacking

Assigning a sequence to several names at once *unpacks* it - this is how Python functions return multiple values. A name prefixed with `*` collects any remaining values into a list.

```
1 def minmax(x):  
2     return min(x), max(x)  
3 minmax([3, 1, 2])
```

py

(1, 3)

```
1 lo, hi = minmax([3, 1, 2])  
2 print(lo, hi)
```

py

1 3

```
1 x, y = 1, 2  
2 x, y = y, x  
3 x, y
```

py

(2, 1)

```
1 first, *rest = range(6)  
2 first
```

py

0

```
1 rest
```

py

[1, 2, 3, 4, 5]

```
1 pairs = [("a", 1), ("b", 2)]  
2 for i, (k, v) in enumerate(pairs):  
3     print(i, k, v)
```

py

0 a 1

1 b 2

The number of names must match the length of the sequence (`x, y = [1, 2, 3]` is a `ValueError`) unless one of them is

Dictionaries

Dictionaries

Python `dicts` are *mutable* containers of **key: value** pairs, designed for the efficient lookup of a value by its key. The closest R analog is a named list (or a named atomic vector when the values are homogeneous).

```
1 {"abc": 123, "def": 456}
```

py

```
{'abc': 123, 'def': 456}
```

```
1 dict([("abc", 123), ("def", 456)])
```

py

```
{'abc': 123, 'def': 456}
```

```
1 dict(abc=123, hello=456)
```

py

```
{'abc': 123, 'hello': 456}
```

```
1 dict(zip(["abc", "def"], [1, 2]))
```

py

```
{'abc': 1, 'def': 2}
```

```
1 list(abc = 123, def = 456)
```

R

```
$abc
```

```
[1] 123
```

```
$def
```

```
[1] 456
```

```
1 c(abc = 123, def = 456)
```

R

```
abc def
```

```
123 456
```

Since Python 3.7 dictionaries preserve insertion order. The keyword form of `dict()` only works for keys that are valid

Keys must be hashable

Dictionary keys and set elements must be **hashable**: their hash must remain stable during their lifetime, and objects that compare equal must have equal hashes. Dictionary values can be anything.

Hashable	Unhashable
<code>int, float, bool, complex</code>	<code>list</code>
<code>str, bytes, range</code>	<code>dict</code>
<code>None, frozenset</code>	<code>set</code>
<code>tuple</code> if all its elements are hashable	<code>tuple</code> containing an unhashable element

Lists are unhashable, so neither a list nor a tuple containing a list can be a key:

```
1 {[1]: "bad"}
```

py

`TypeError: cannot use 'list' as a dict key`

```
1 {(1, [2]): "bad"}
```

py

`TypeError: cannot use 'tuple' as a dict key`

`hash()` returns an integer used to locate an entry in a hash table. Hashability is distinct from immutability: instances of user-defined classes are hashable by identity by default, even when their attributes can change. Defining `__eq__` in a

Lookup

[] raises a `KeyError` for a missing key, `in` tests for a key and `.get()` supplies a default.

```
1 x = {1: "abc", "y": "hello", (1, 1): 3.14}
2 x[1]
```

'abc'

```
1 x[(1, 1)]
```

3.14

```
1 x["def"]
```

KeyError: 'def'

```
1 "y" in x
```

True

```
1 "hello" in x
```

False

```
1 print( x.get("def") )
```

None

```
1 x.get("def", 0)
```

0

Insert, replace, remove

Assigning to a key inserts or replaces, `del` and `.pop()` remove. Note that assigning `None` does *not* remove a key, whereas in R assigning `NULL` does.

```
1 x = {1: "abc", "y": "hello"}
2 x["def"] = -1
3 x["y"] = "goodbye"
4 x
```

py

```
{1: 'abc', 'y': 'goodbye', 'def': -1}
```

```
1 del x[1]
2 x.pop("def")
```

py

```
-1
```

```
1 x["y"] = None
2 x
```

py

```
{'y': None}
```

```
1 y = list(abc = 123, def = 456)
2 y$ghi = -1
3 y$def = "goodbye"
4 str(y)
```

R

```
List of 3
 $ abc: num 123
 $ def: chr "goodbye"
 $ ghi: num -1
```

```
1 y$abc = NULL
2 str(y)
```

R

```
List of 2
 $ def: chr "goodbye"
 $ ghi: num -1
```

Methods & iteration

Iterating over a `dict` yields its *keys* - use `.items()` to get keys and values together.

```
1 x = {1: "abc", "y": "hello"}
2 x.keys()
```

py

`dict_keys([1, 'y'])`

```
1 x.values()
```

py

`dict_values(['abc', 'hello'])`

```
1 x.items()
```

py

`dict_items([(1, 'abc'), ('y', 'hello')])`

```
1 for k, v in x.items():
2     print(k, "->", v)
```

py

1 -> abc
y -> hello

```
1 x | {"y": "goodbye", "w": 0}
```

py

`{1: 'abc', 'y': 'goodbye', 'w': 0}`

```
1 x.update({"y": "goodbye", "w": 0})
2 x
```

py

`{1: 'abc', 'y': 'goodbye', 'w': 0}`

`.keys()`, `.values()`, and `.items()` return dynamic views of the dictionary rather than copies - see the [docs](#). `|` returns a

Environments

Environments

An environment is a set of name-object *bindings* - the workspace is one, each package is one, and every function call creates a fresh one for its local variables. A lookup that fails in an environment continues in its *parent*.

```
1 x = 1
2 f = function() {
3   y = 2
4   environment()
5 }
6 e = f()
7 ls(e)
```

R

```
[1] "y"
```

```
1 parent.env(e)
```

R

```
<environment: R_GlobalEnv>
```

```
1 get("x", envir = e)
```

R

```
[1] 1
```

```
1 g = function() {
2   z = 3
3   z
4 }
5 g()
```

R

```
[1] 3
```

```
1 exists("z")
```

R

```
[1] FALSE
```

`globalenv()` is the workspace and the chain of parents ends at `emptyenv()`. Looking up through the parents is *lexical scoping* - `f()` finds `x` in the global environment, and `z` is gone once `g()` returns. More on this with functional

Environments as hash maps

Bindings are stored in a hash table, so an environment makes a good `dict` - use `parent = emptyenv()` so that lookups do not fall through to the workspace.

```
1 h = new.env(parent = emptyenv())
2 h$a = 1
3 h[["b"]] = 2
```

```
1 ls(h)
```

```
[1] "a" "b"
```

```
1 h$a
```

```
[1] 1
```

```
1 exists("x", envir = h)
```

```
[1] FALSE
```

```
1 rm("a", envir = h)
2 ls(h)
```

```
[1] "b"
```

```
1 h2 = h
2 h2$d = 4
3 ls(h)
```

```
[1] "b" "d"
```

```
1 h[1]
```

Error in `h[1]`:
! object of type 'environment' is not subsettable

Modifying `h2` changed `h` - environments have *reference semantics*, both names refer to the same object exactly like two

Exercise 1

Using the record from last lecture, complete these tasks in Python:

```
1 person = {
2     "firstName": "John",
3     "lastName": "Smith",
4     "age": 25,
5     "address": {
6         "streetAddress": "21 2nd Street",
7         "city": "New York",
8         "state": "NY",
9         "postalCode": 10021
10    },
11    "phoneNumber": [
12        { "type": "home",
13          "number": "212 555-1239" },
14        { "type": "fax",
15          "number": "646 555-4567" }
16    ]
17 }
```

PY

- Extract the fax number.
- Add a "mobile" phone number and remove age.
- Write `phone(person, phone_type)` to return the requested number, or `None` if absent.
- **Extension:** Write the equivalent function in R.

Sets

Sets

A **set** is a *mutable, unordered* collection of **unique** hashable elements - there are no positions, so the primary operation is membership testing with **in**.

```
1 x = {1, 2, 3, 4, 1, 2}
2 x
```

py

```
{1, 2, 3, 4}
```

```
1 set("mississippi")
```

py

```
{'i', 'p', 's', 'm'}
```

```
1 3 in x
```

py

```
True
```

```
1 x[0]
```

py

```
TypeError: 'set' object is not subscriptable
```

```
1 x.add(9)
2 x.discard(8)
3 x.update([7, 8])
4 x
```

py

```
{1, 2, 3, 4, 7, 8, 9}
```

```
1 x.remove(6)
```

py

```
KeyError: 6
```

```
1 {1, 2, [1, 2]}
```

py

```
TypeError: cannot use 'list' as a set element (u
```

`{}` creates an empty *dictionary*, an empty set must be written `set()` `remove()` raises a **KeyError** if the element is absent

Set operations

Sets support the usual mathematical operations. R has no set type - any vector can be treated as one via `unique()` and the set functions.

```
1 x = {1, 2, 3}
2 y = {2, 3, 4}
3 x | y
```

py

{1, 2, 3, 4}

```
1 x & y
```

py

{2, 3}

```
1 x - y
```

py

{1}

```
1 1 in x
```

py

True

```
1 x = c(1, 2, 3, 1)
2 y = c(2, 3, 4)
3 union(x, y)
```

R

[1] 1 2 3 4

```
1 intersect(x, y)
```

R

[1] 2 3

```
1 setdiff(x, y)
```

R

[1] 1

```
1 1 %in% x
```

R

[1] TRUE

Python's set methods (`x.union(y)`, `x.intersection(y)`, ...) accept any iterable, the operators require sets on both sides.

Mutability & copies

Nested containers

In Lecture 2 we saw that `.copy()` is *shallow*, and `copy.deepcopy()` copies recursively. The same sharing problem occurs when building nested structures by repetition,

```
1 grid = [[0] * 3] * 3
2 grid[0][0] = 1
3 grid
```

py

```
[[1, 0, 0], [1, 0, 0], [1, 0, 0]]
```

```
1 grid = [[0] * 3 for _ in range(3)]
2 grid[0][0] = 1
3 grid
```

py

```
[[1, 0, 0], [0, 0, 0], [0, 0, 0]]
```

```
1 import copy
2
3 x = {"a": [1, 2], "b": [3, 4]}
4 y = x.copy()
5 z = copy.deepcopy(x)
6
7 x["a"].append(99)
8 y
```

py

```
{'a': [1, 2, 99], 'b': [3, 4]}
```

```
1 z
```

py

```
{'a': [1, 2], 'b': [3, 4]}
```

`[[0] * 3] * 3` creates *one* inner list and references it three times. The comprehension evaluates `[0] * 3` three separate

Mutable default arguments

Default values are evaluated *once*, when the function is defined, so a mutable default is shared by every call that does not supply the argument. Use `None` as the default and create the object inside the function instead.

```
1 def f(x=[]):  
2     x.append(1)  
3     return x  
4  
5 f()
```

py

[1]

```
1 f()
```

py

[1, 1]

```
1 def f(x=None):  
2     if x is None:  
3         x = []  
4     x.append(1)  
5     return x  
6  
7 f()
```

py

[1]

```
1 f()
```

py

[1]

Algorithms & data structures

Big-O notation

Big-O notation describes the *complexity* of an algorithm - how the time (or memory) required grows with the size of the input n . Only the fastest growing term matters (constant factors and lower order terms are dropped), so two algorithms with the same Big-O can still differ greatly in practice.

Since performance depends on the data we usually quote *average* or *worst case* complexity, and *amortized* complexity when an occasional expensive step is paid for by many cheap ones (e.g. growing a list).

Complexity	Big-O
Constant	$O(1)$
Logarithmic	$O(\log n)$
Linear	$O(n)$
Quasilinear	$O(n \log n)$
Quadratic	$O(n^2)$
Exponential	$O(C^n)$

Under the hood

The containers in both languages are built from a handful of basic structures - knowing which is which explains what each container is good, and bad, at.

Structure	Layout	R	Python
Array	contiguous block of same-sized values	materialized numeric vector	contiguous numeric <code>ndarray</code>
Array of pointers	contiguous block of <i>references</i> to objects	list	<code>list</code> , <code>tuple</code>
Linked list	nodes that each point to their neighbors	pairlist	<code>deque</code> (linked blocks)
Hash table	array indexed by <code>hash(key)</code>	environment	<code>dict</code> , <code>set</code>

Vectors

Materialized numeric vectors in R and contiguous numeric NumPy arrays store fixed-size values together, giving $O(1)$ element access. NumPy views can have gaps or reversed strides.

```
1 object.size(numeric(1e6))
```

R

8000048 bytes

```
1 object.size(integer(1e6))
```

R

4000048 bytes

```
1 np.zeros(1_000_000).nbytes
```

py

8000000

```
1 np.zeros(1_000_000, dtype="int64").nbytes
```

py

8000000

```
1 np.zeros(1_000_000, dtype="int32").nbytes
```

py

4000000

R doubles use 8 bytes per element; integers use 4. `object.size()` includes object overhead; NumPy's `.nbytes` counts

Growing a vector

Repeatedly appending with `c(x, i)` copies the existing elements each time, giving $O(n^2)$ work overall.

```
1 n = 5e4
2 x = c()
3 system.time(
4   for (i in seq_len(n)) x = c(x, i)
5 )
```

user	system	elapsed
1.050	0.020	1.071

Indexed growth can reuse spare capacity, but still needs occasional copying. Preallocation avoids repeated growth.

```
1 x = numeric()
2 system.time(
3   for (i in seq_len(n)) x[i] = i
4 )
```

user	system	elapsed
0.004	0.000	0.004

```
1 x = numeric(n)
2 system.time(
3   for (i in seq_len(n)) x[i] = i
4 )
```

user	system	elapsed
0.002	0.000	0.001

Generic vectors

R and Python lists store references to objects. Elements can have different types and sizes, while indexing remains $O(1)$. References and object headers add memory overhead.

```
1 Numeric vector: values stored together
2
3 [ 1.0 | 2.0 | 3.0 ]
4
5
6 List: references stored together
7
8 [ • | • | • ]
9   ↓   ↓   ↓
10  1  "abc" TRUE
```

Array



<https://visualgo.net/en/array> - Materialized numeric R vectors and contiguous numeric NumPy arrays store same-typed values together. A Python `list` is a contiguous array of *pointers* to objects, which is why it can be heterogeneous and

deques

A **deque** (double-ended queue) supports $O(1)$ additions and removals at either end. CPython stores it as a doubly linked chain of fixed-size blocks. Indexing is $O(1)$ near either end and $O(n)$ in the middle.

```
1 from collections import deque
2 x = deque(range(3))
3 x.appendleft(-1)
4 x
```

py

```
deque([-1, 0, 1, 2])
```

```
1 x.popleft()
```

py

```
-1
```

```
1 x
```

py

```
deque([0, 1, 2])
```

```
1 x = deque(range(3), maxlen=4)
2 x.append(10)
3 x.append(11)
4 x
```

py

```
deque([1, 2, 10, 11], maxlen=4)
```

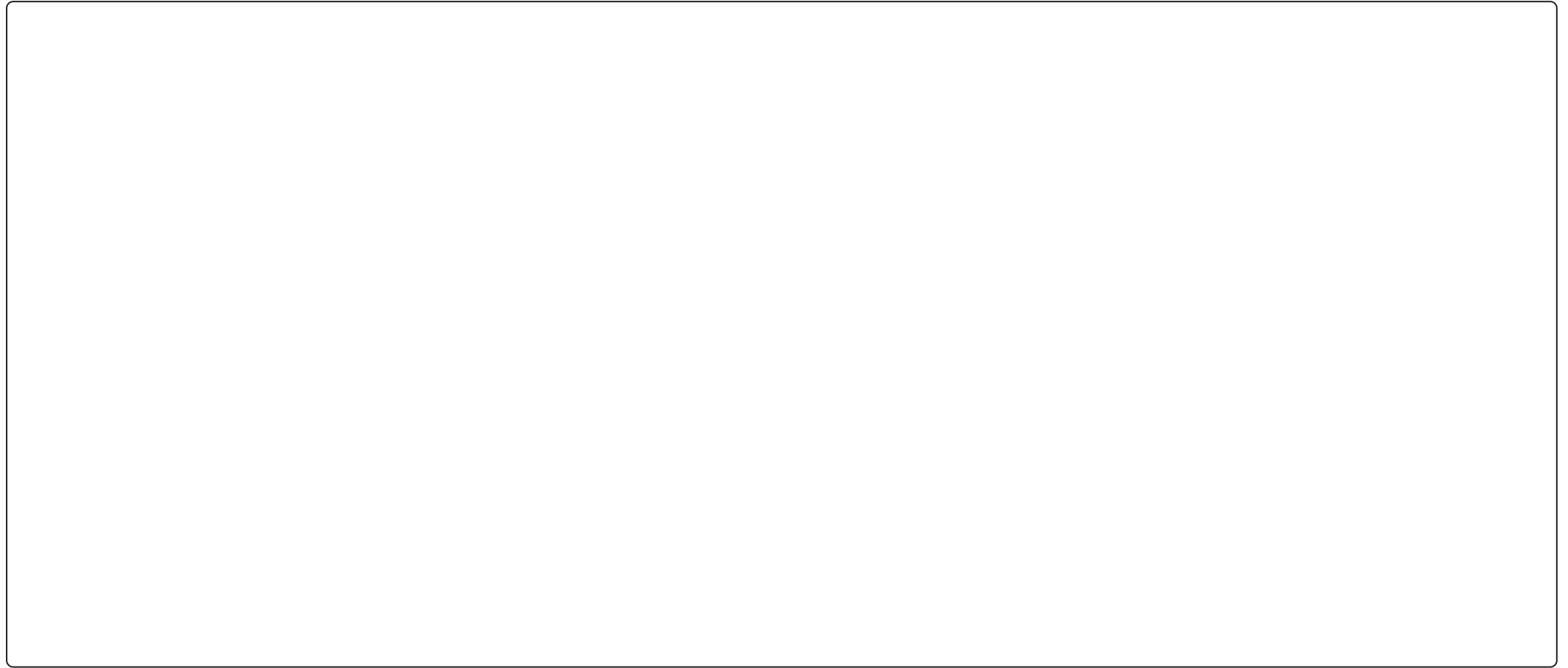
```
1 x.appendleft(-1)
2 x
```

py

```
deque([-1, 1, 2, 10], maxlen=4)
```

With **maxlen** set, adding to a full deque drops an element from the opposite end - a rolling window of the last n items.

Linked list



Hash tables in practice

Hash-table membership is $O(1)$ on average; searching a list is $O(n)$. These timings illustrate the practical difference:

```
1 import time
2 big_list = list(range(100_000))
3 big_set = set(big_list)
4 queries = range(0, 100_000, 100)
5
6 start = time.perf_counter()
7 res = [q in big_list for q in queries]
8 print(time.perf_counter() - start)
```

py

0.20772770800977014

```
1 start = time.perf_counter()
2 res = [q in big_set for q in queries]
3 print(time.perf_counter() - start)
```

py

0.0014382079971255735

```
1 big = sample(1e5)
2 queries = sample(1e5, 1e3)
3
4 system.time({
5   for (q in queries) any(big == q)
6 })
```

R

user	system	elapsed
0.111	0.000	0.111

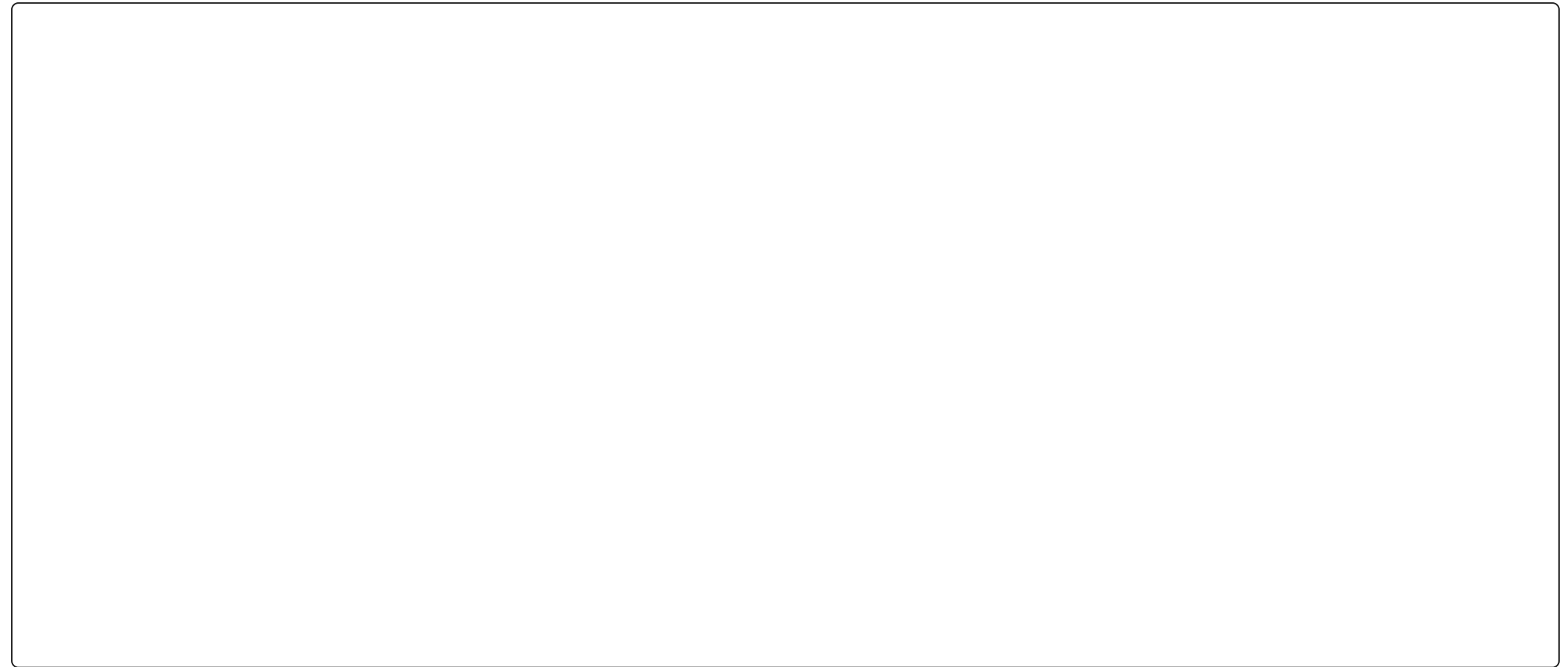
```
1 system.time({
2   queries %in% big
3 })
```

R

user	system	elapsed
0.005	0.000	0.005

Python's set is built before timing; R's `%in%` includes lookup preparation. To study scaling, vary the container size while

Hash table



<https://visualgo.net/en/hashtable> - Python's `dict` and `set` are hash tables, as are R environments. R's `match()`, `%in%`, `unique()`, and `duplicated()` build a hash table of one argument, which is why they are fast. Equal keys must hash

Sequence operation costs

Operation	<code>list</code>	<code>deque</code>
Copy	<code>.copy(): O(n)</code>	<code>.copy(): O(n)</code>
Get / replace by index	<code>s[i] / s[i] = v: O(1)</code>	<code>s[i] / s[i] = v: O(n)</code>
Append at end	<code>.append(v): O(1)*</code>	<code>.append(v): O(1)</code>
Remove from end	<code>.pop(): O(1)*</code>	<code>.pop(): O(1)</code>
Insert at front	<code>.insert(0, v): O(n)</code>	<code>.appendleft(v): O(1)</code>
Remove from front	<code>.pop(0): O(n)</code>	<code>.popleft(): O(1)</code>
Insert / delete in middle	<code>O(n)</code>	<code>O(n)</code>
Membership	<code>v in s: O(n)</code>	<code>v in s: O(n)</code>

CPython costs for n elements. *List append and end-pop costs are amortized. Deque indexing is $O(1)$ near either end and

Dictionary and set operation costs

Operation	<code>dict</code>	<code>set</code>
Copy	<code>.copy(): O(n)</code>	<code>.copy(): O(n)</code>
Lookup value	<code>d[k]: O(1)</code>	—
Insert / update	<code>d[k] = v: O(1)</code>	<code>.add(v): O(1)</code>
Remove	<code>.pop(k): O(1)</code>	<code>.remove(v): O(1)</code>
Membership	<code>k in d: O(1)</code>	<code>v in s: O(1)</code>

Hash-table operations are $O(1)$ on average, assuming constant cost hashing and equality; individual operations can take

Exercise 2

Choose two scenarios. For each, suggest a data structure in Python and R. Explain which operations your choice makes efficient, and state any assumptions.

- A collection of 100 integers used repeatedly in elementwise arithmetic.
- A queue (first in, first out) of customer records.
- A stack (first in, last out) of customer records.
- A count of word occurrences within a document.
- The heights of the bars in a histogram with even bin widths.
- Checking whether each of a million words is in a list of 50 stop words.
- The last 10 lines read from a very large log file.

Comparing R & Python

Container summary

Python	Properties	R
<code>list</code>	ordered, mutable, heterogeneous	<code>list()</code>
<code>tuple</code>	ordered, immutable, heterogeneous	<code>list()</code> or atomic vector (copy on modify)
<code>dict</code>	key lookup, mutable, insertion ordered	named <code>list()</code> , <code>new.env()</code> for hashing
<code>set</code>	unique, unordered, mutable	vector + <code>unique()</code> , <code>union()</code> , <code>%in%</code> , ...
<code>deque</code>	fast at both ends	none (use a list or vector)
<code>range</code>	lazy integer sequence, immutable	<code>seq_len()</code> , <code>:</code> (often stored compactly via ALTREP)
NumPy <code>ndarray</code>	homogeneous, vectorized	atomic vector, matrix, array

Takeaways

- **Tuples** are immutable sequences—use them for fixed groups of values and unpacking.
- **Dictionaries** map hashable keys to values, with $O(1)$ lookup on average. R offers named lists and hash-based environments.
- **Sets** provide $O(1)$ membership on average and set algebra. In R, use `%in%`, `union()`, `intersect()`, and `setdiff()`.
- **Shallow copies** share nested objects. Use `deepcopy()` when nested objects need independent mutation.
- **Mutable defaults** are shared between calls. Use `None` and create the object inside the function.
- Choose containers for their operations: deques for queues, sets for membership, and preallocated R vectors when the size is known. Measure when performance matters.